

YENEPOYA
(DEEMED TO BE UNIVERSITY)
Project –High Level Design
on
Personalized Recommendation Engine for Retail

Student Name: Niha Najeeb

Industry Mentor: Sumit Shukla

Guided By: Ankitha

Table of Contents

1. Introduction.	6
1.1. Scope of the document.	6
1.2. Intended Audience	6
1.3. System overview.	6
2. System Design.	6
2.1. Application Design	6
2.2. Process Flow.	6
2.3. Information Flow.	7
2.4. Components Design	8
2.5. Key Design Considerations	8
2.6. API Catalogue.	8
3. Data Design.	8
3.1. Data Model	8
3.2. Data Access Mechanism	9
3.3. Data Retention Policies	9
3.4. Data Migration	9
4. Interfaces	9
5. State and Session Management	10
6. Caching	10
7. Non-Functional Requirements	10
7.1. Security Aspects	10
7.2. Performance Aspects	10
8. References	11

1. Introduction

1.1 Scope of the document

This document presents the High-Level Design (HLD) of the Retail Recommendation Dashboard. It provides a comprehensive overview of the system architecture, major components, data flow, and design considerations involved in building a content-based recommendation system for retail products.

- The scope includes:
- System architecture and module design
- Data processing and recommendation workflow
- User interface structure
- Functional and non-functional requirements

1.2 Intended Audience

This document is intended for:

- Software developers and system designers
- Academic evaluators and project reviewers
- Students implementing AI/ML-based systems
- Stakeholders interested in system functionality

1.3 System Overview

The Retail Recommendation Dashboard is an intelligent system that suggests similar products based on product attributes such as category, usage, and features.

The system uses a **content-based filtering approach**, where each product is represented by a set of descriptive tags. These tags are converted into numerical vectors, and cosine similarity is used to measure the closeness between products.

The dashboard provides an interactive interface where users can select a product and view similar items displayed in a card-based layout, including product details such as price and rating.

2. System Design

2.1 Application Design

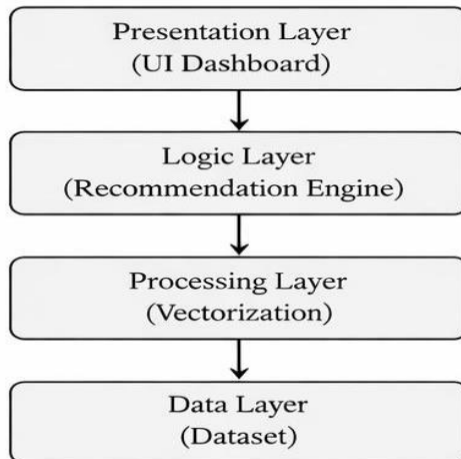


Figure 2.1: Application Design Diagram

The system follows a layered modular architecture consisting of:

- **Data Layer:**
Stores product dataset including item name, tags, price, and rating.
- **Processing Layer:**
Handles preprocessing of textual data such as cleaning and formatting tags.
- **Feature Extraction Layer:**
Converts text data into numerical vectors using CountVectorizer.
- **Logic Layer:**
Computes similarity scores using cosine similarity.
- **Presentation Layer:**
Displays recommendations through an interactive dashboard using widgets and HTML rendering.

2.2 Process Flow



Figure 2.2: Process Flow Diagram

The process flow of the system describes how input data is processed to generate recommendations:

1. The product dataset is loaded into the system
2. Textual features (tags) are preprocessed
3. Tags are converted into numerical vectors
4. Cosine similarity is calculated between all products
5. User selects a product from the dashboard
6. System retrieves similar products based on similarity scores
7. Recommended products are displayed in card format

2.3 Information Flow

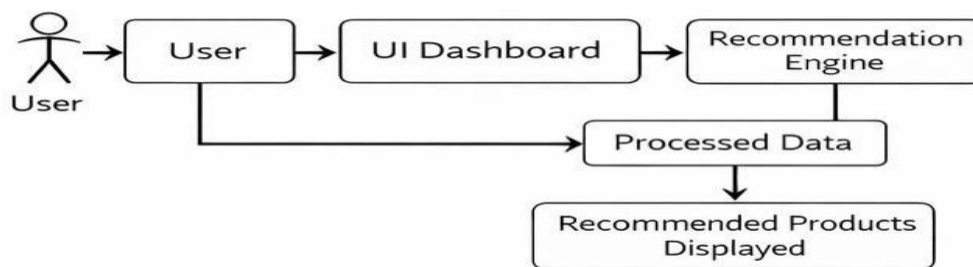


Figure 2.3: Information Flow Diagram

The information flow in the system is as follows:

- User interacts with the dashboard and selects a product
- Input is passed to the recommendation engine
- The system retrieves the corresponding feature vector

- Similarity computation is performed
- Top matching products are selected
- Results are passed to the UI layer
- UI displays products with price, rating, and similarity score

2.4 Components Design

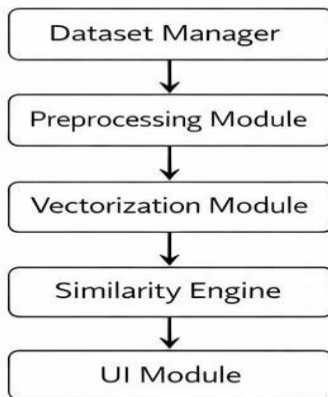


Figure 2.4: Components Design Diagram

The system is divided into the following major components:

- **Dataset Manager:**
Responsible for storing and managing product data including tags, price, and rating.
- **Preprocessing Module:**
Cleans and standardizes textual data for consistent processing.
- **Vectorization Module:**
Converts text data into numerical vectors for similarity computation.
- **Similarity Engine:**
Computes similarity scores between products using cosine similarity.
- **Recommendation Module:**
Selects and ranks the most similar products.
- **UI Module:**
Displays recommendations in a card-based shopping interface using interactive widgets.

2.5 Key Design Considerations

- The system is designed with the following considerations:

- Simplicity and ease of use
- Efficient computation of similarity
- Scalability for larger datasets
- Modular architecture for easy updates
- Interactive and visually appealing user interface

2.6 API Catalogue

API Name	Description	Input	Output
get_products()	Retrieves all products	None	List of products
recommend_product()	Provides recommendations	Product name	List of similar products
add_product()	Adds new product	Product details	Status
get_product_details()	Fetch product info	Product ID	Product data

3. Data Design

3.1 Data Model

Field Name	Type	Description
item	String	Product name
tags	Text	Product features
price	Integer	Product price
rating	Float	Product rating

3.2 Data Access Mechanism

- Data is stored in a Pandas DataFrame and accessed using Python functions. Data retrieval and processing are performed in-memory for faster computation.

3.3 Data Retention Policies

- The system does not store sensitive user data. Product data is stored locally and can be updated or replaced as needed. Data retention is flexible based on system requirements.

3.4 Data Migration

The dataset can be imported from CSV files and can be extended to relational or NoSQL databases in future implementations.

4. Interfaces

The system includes the following interfaces:

- **User Interface:**
Interactive dashboard built using Jupyter widgets and HTML styling.
- **System Interface:**
Communication between modules using Python functions.
- **External Interface:**
Future integration with APIs such as product databases or recommendation services.

5. State and Session Management

The system operates in a stateless manner. Each request is processed independently without maintaining user sessions or history.

6. Caching

Currently, caching is not implemented. However, similarity matrices can be cached in memory to improve performance in future versions.

7. Non-Functional Requirements

7.1 Security Aspects

- No personal or sensitive data is handled

- Input validation to prevent errors
- Secure local execution environment

7.2 Performance Aspects

- Fast execution using vectorized operations
- Efficient for small and medium datasets
- Minimal memory and processing overhead

8. References

- Python Official Documentation
- Pandas Documentation
- Scikit-learn Documentation
- Research papers on recommender systems
- Online tutorials and academic resources